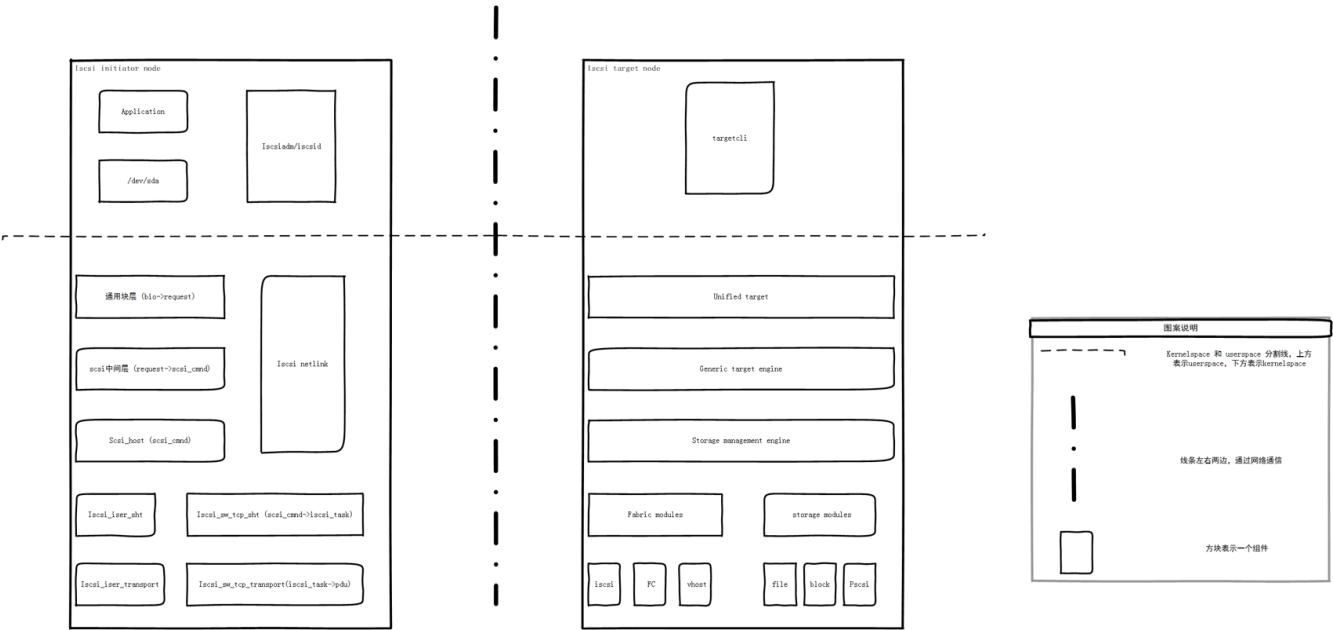


ISCSI 架构介绍



ISCSI 整体架构如上图所示，主要分为initiator 节点和target 节点，下面会整体介绍一下怎么工作，具体每个组件细节会一一展示。

(1) target 节点：

target 节点主要分为两个部分，用户态工具targetcli，和内核态组件LIO-target(TCM)，targetcli 通过configs 对LIO-target进行管理，比如：创建删除tpg，添加删除lun等。

Lio-target是真正提供iscsi 服务的模块，其中fabric modules 包括各种支持的协议，包括iscsi，fc，vhost等。storage modules 是对接后端存储的模块，同样是插件式设计，可以支持各种不同类型的backstore类型，包括block，file，pscsi等。

IO 通过fabric模块接收之后，进行解析处理，最后映射到storage模块发送给对应的backstore进行实际IO操作。

(2) initiator 节点：

Initiator 主要也是两个部分，一部分是用户态open-iscsi项目（iscsiadm和iscsid），另一部分是iscsi的本地驱动，iscsi的本地驱动属于scsi驱动的一部分。open-iscsi 通过iscsi_netlink机制和iscsi驱动进行交互，比如：创建session，创建connection等。同时也会使用其他通用的交互方式和SCSI进行交互，比如，通过sysfs来通知其scan target。

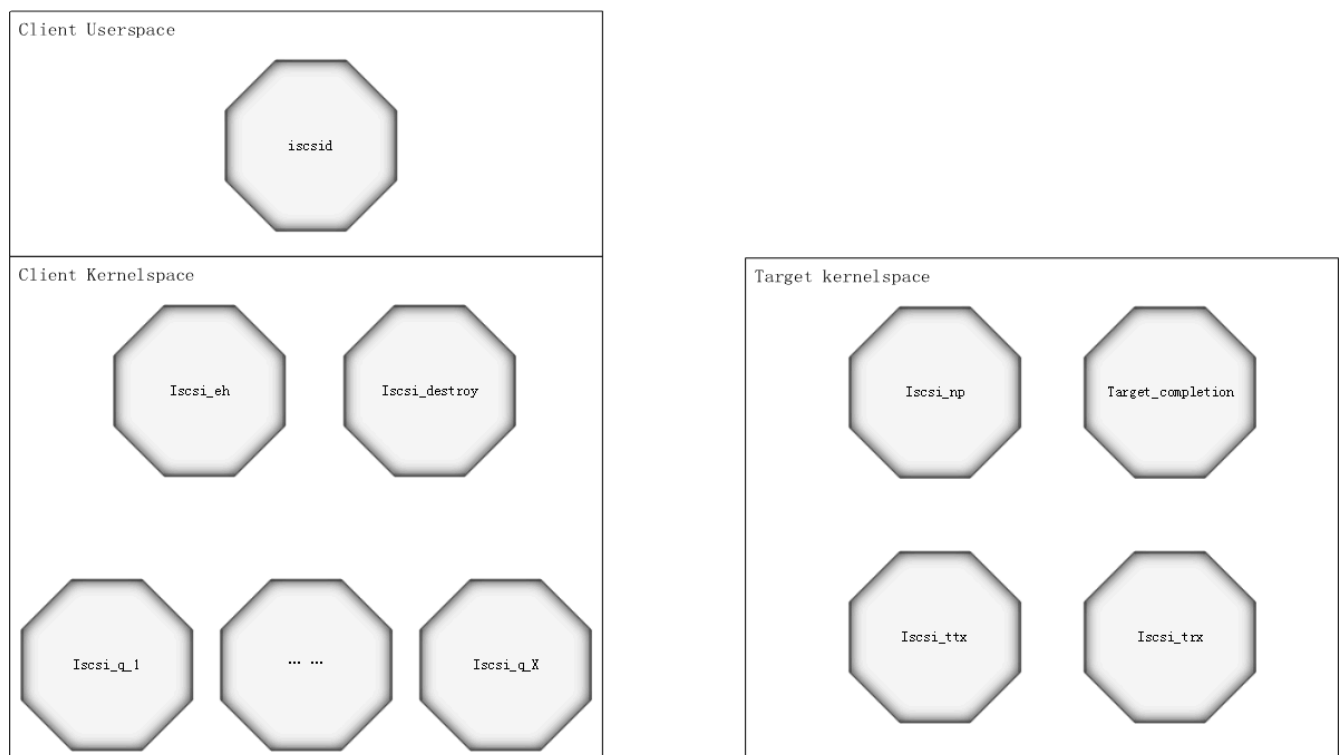
iscsiadm 是一个命令最常用的有三个操作：discovery，login，logout。他会通过AF_LOCAL socket 和iscsid进行交互，也会直接通过网络连接target，比如发送sendtargets来做discovery。

iscsid 是一个服务，他会监听iscsiadm 通过AF_LOCAL socket

"ISCSIADM_ABSTRACT_NAMESPACE"发过来的信息。也会通过netlink和kernel driver进行交互，比如create_session。

initiator节点需要考察几个重点是：（1）如何discovery，（2）如何挂载 lun （3）IO流程 （4）网络异常如何处理。

前文通过逻辑视图介绍了基于LIO的iscsi架构，下面可以通过进程视图再来看看各进程之间的职责划分和交互方式。



图中每一个正八边形表示一个进程。

client端分为用户态和内核态：

用户态：

client端的iscsid 进程服务会一直存在，承担管理工作。

内核态：

client端的iscsi_eh 是一个内核进程，是一个iscsi的error handler，承担错误处理的一些工作。

client端的iscsi_destroy也是一个内核进程，在destroy session的时候会承担异步destroy工作。

client端的iscsi_q_x是一个内核进程，这个进程是每一个iscsi device一个，他主要承担数据发送工作Xmit_work。

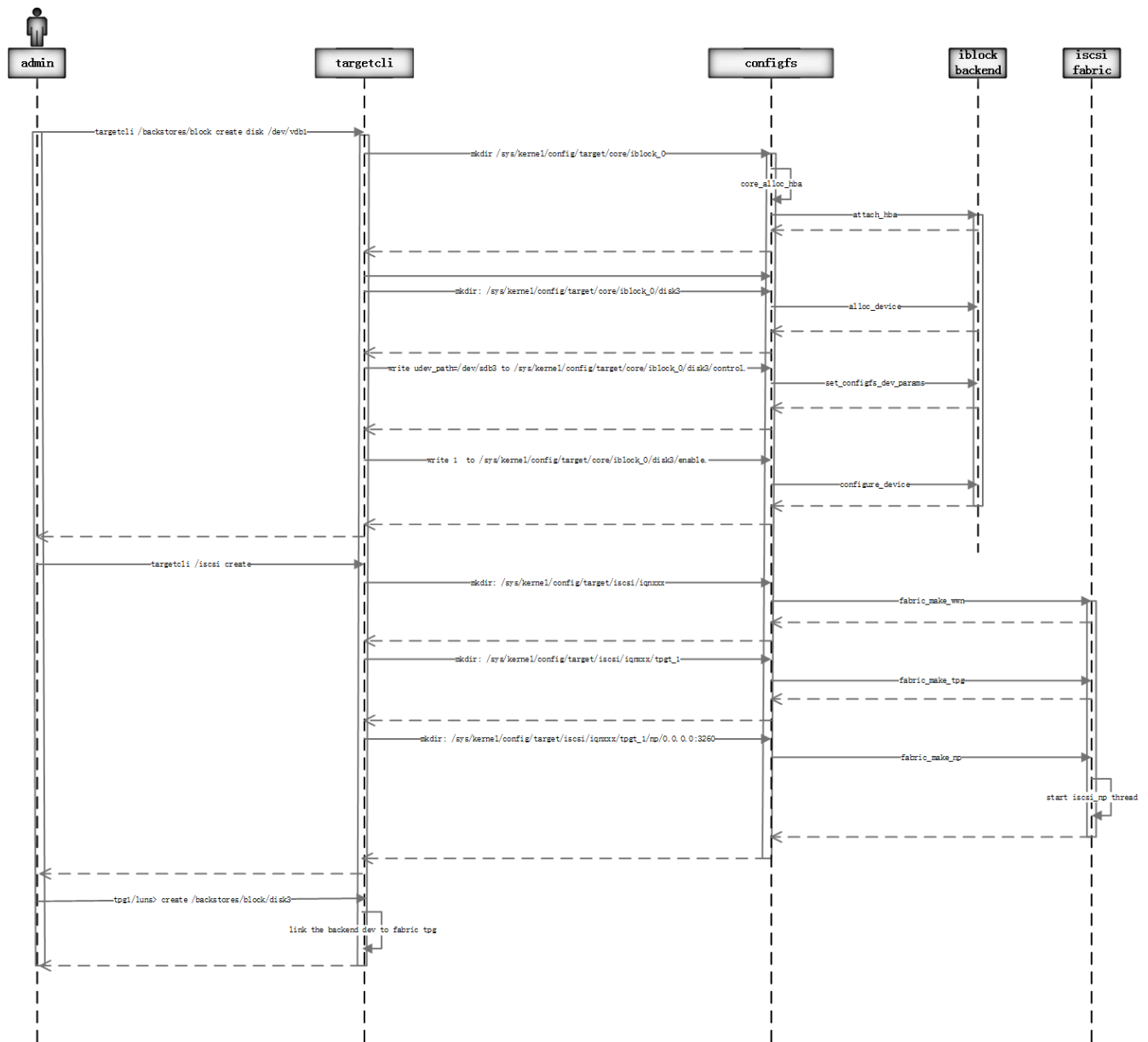
target端进程主要都在内核态，用户态没有常驻进程，只是在配置阶段会有targetcli命令用于配置而已。

iscsi_np 是为networkportal对应的一个进程，是一个入口进程，当一个tpg绑定了np之后，就可以通过这个np访问到tpg了。默认np：0.0.0.0:3260。也可以使用其他np。

target_completion是一个workqueue，target cmd 完成的时候通过这个workqueue来完成。

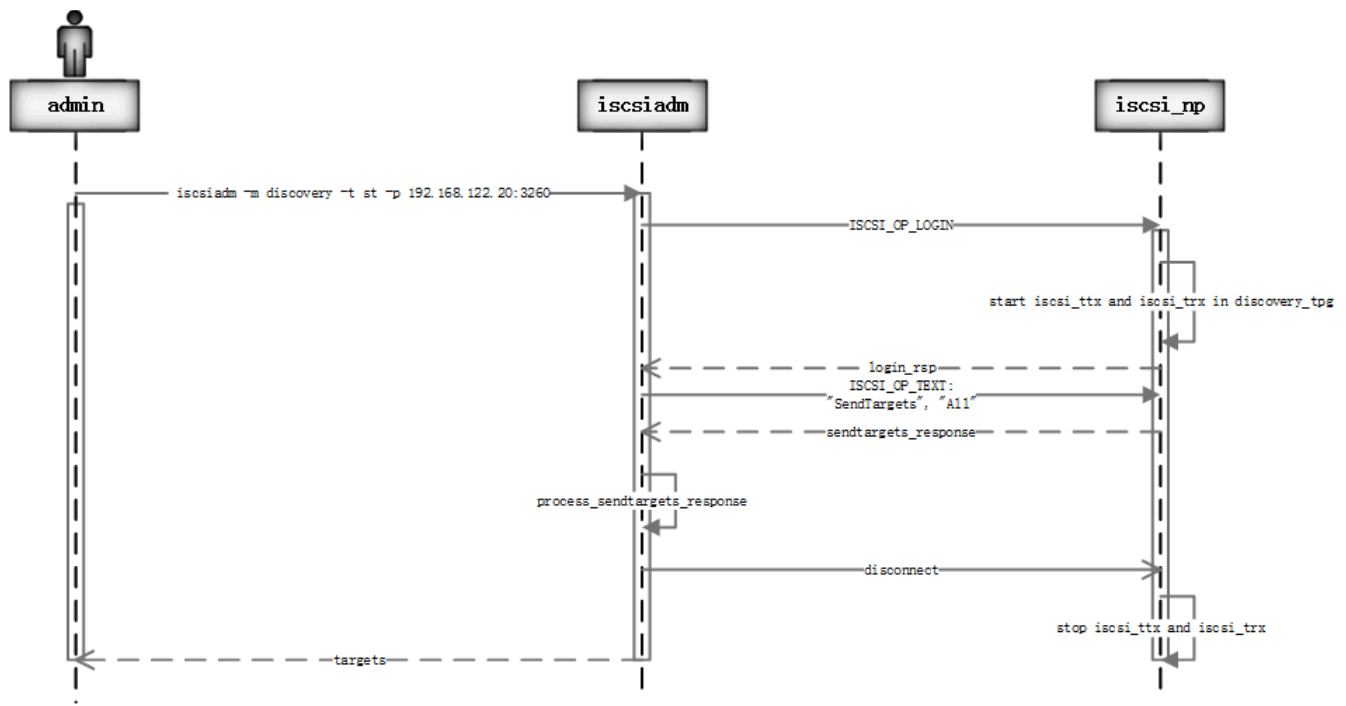
iscsi_ttx和iscsi_trx是每一个tpg相关联的两个进程。主要负责transmit和receive。

Target setup:



targetcli 通过configfs和LIO系统进行交互，三个阶段：（1）添加backstore，（2）添加tpg，并启动np（3）绑定backstore到tpg的lun。

Discovery:



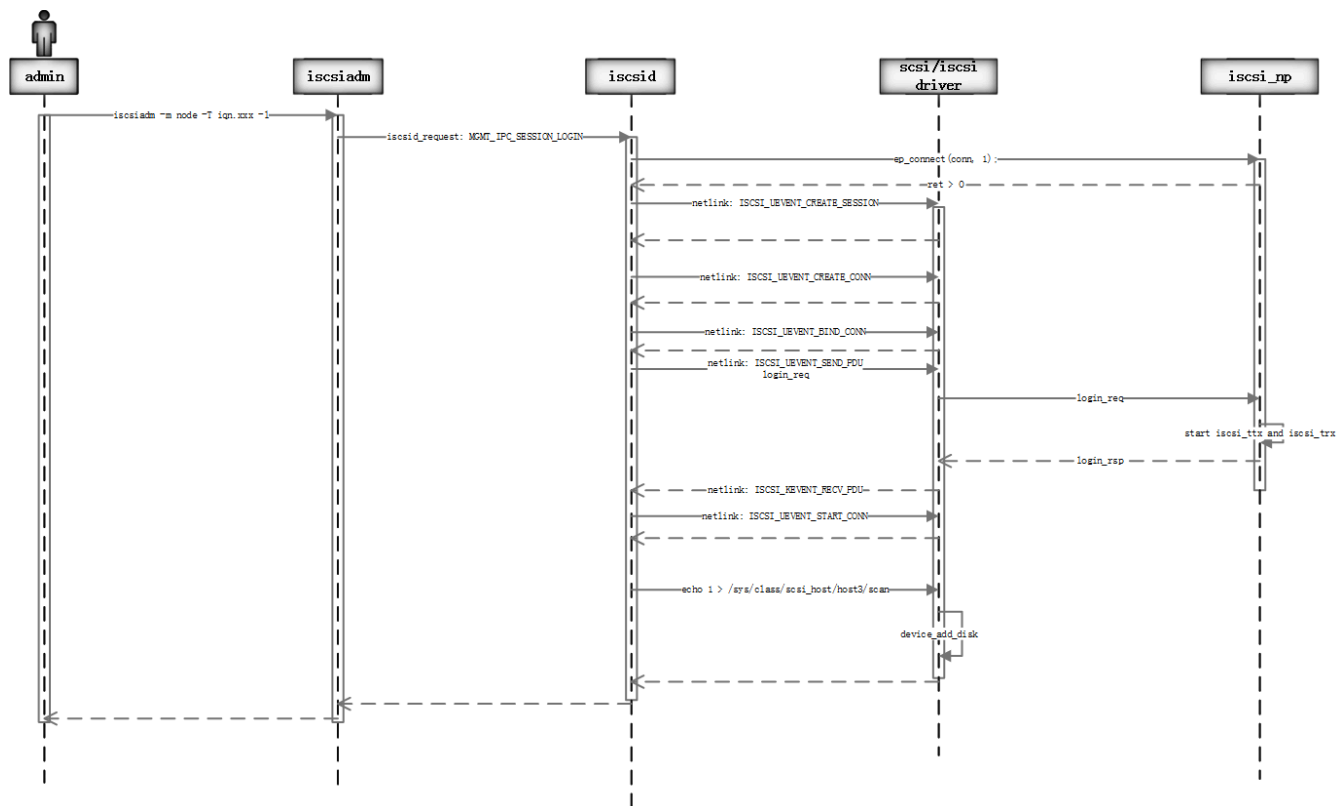
discovery 主要是iscsiadm 和target 进行交互。

(1) iscsiadm 向target 发送login request，target会启动两个全局的进程iscsi_ttx和iscsitrx，这两各进程在discovery_tpg里面。

(2) iscsiadm 在发送一个sendtargets text request 到 target的iscsi_trx，会返回所有的target信息。

(3) iscsiadm 解析了sendtarget response，打印出来。最后disconnect掉连接。target端检测到网络断开，stop掉ttx和trx。

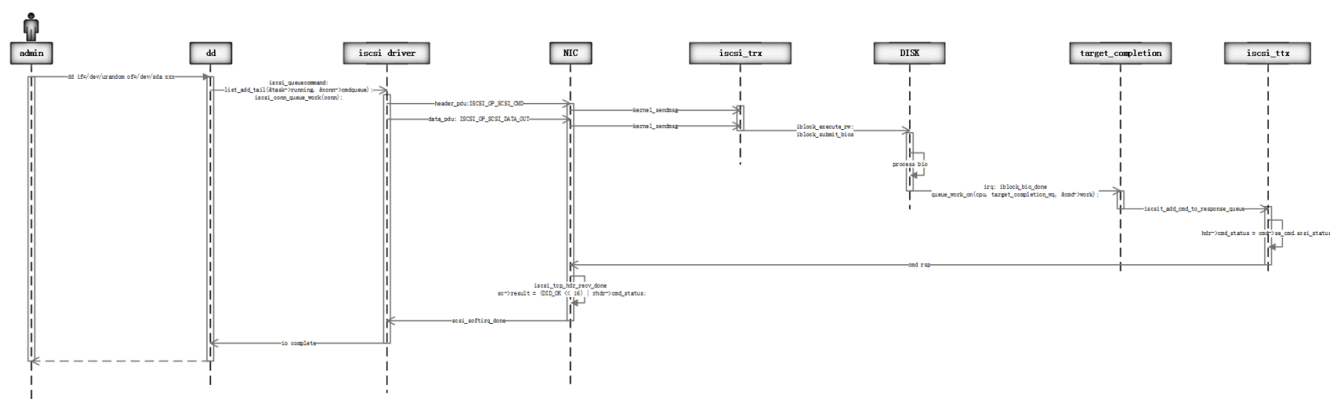
Login:



Login 比discovery 复杂得多，login的流程不再是iscsiadm直接去做的，而是通过socket通知iscsid去完成整个流程。

- (1) iscsid 首先从userspace 去connect target，如果不成功，返回失败；如果成功，继续（2）。
- (2) iscsid已经探测到target 是available的，所以通过netlink通知kernel开始连接，create session, create connect, bind connect。
- (3) 然后iscsid 通过netlink 让kernel 发送login request 给target。target启动对应的iscsi_ttx，和 iscsi_trx。
- (4) target 返回login response给kernel，kernel将login_rsp 返回给iscsid，iscsid 解析结果。
- (5) iscsid 如果发现login 成功，会想kernel 发送一个start connection netlink，这个是告诉kernel connection可以开始正常工作了。
- (6) 最后iscsid 会通过sysfs 来scan 这个iscsi host，会probe lun，然后添加disk。这里面会发送 INQUIRY到target去探测信息。如果探测成功，就会添加一个disk到 devtmpfs： device_add_disk->register_disk->device_add->devtmpfs_create_node，并发送一个disk_uevent(disk, KOBJ_ADD); 到udev，然后由/lib/udev/rules.d/60-persistent-storage.rules 处理。

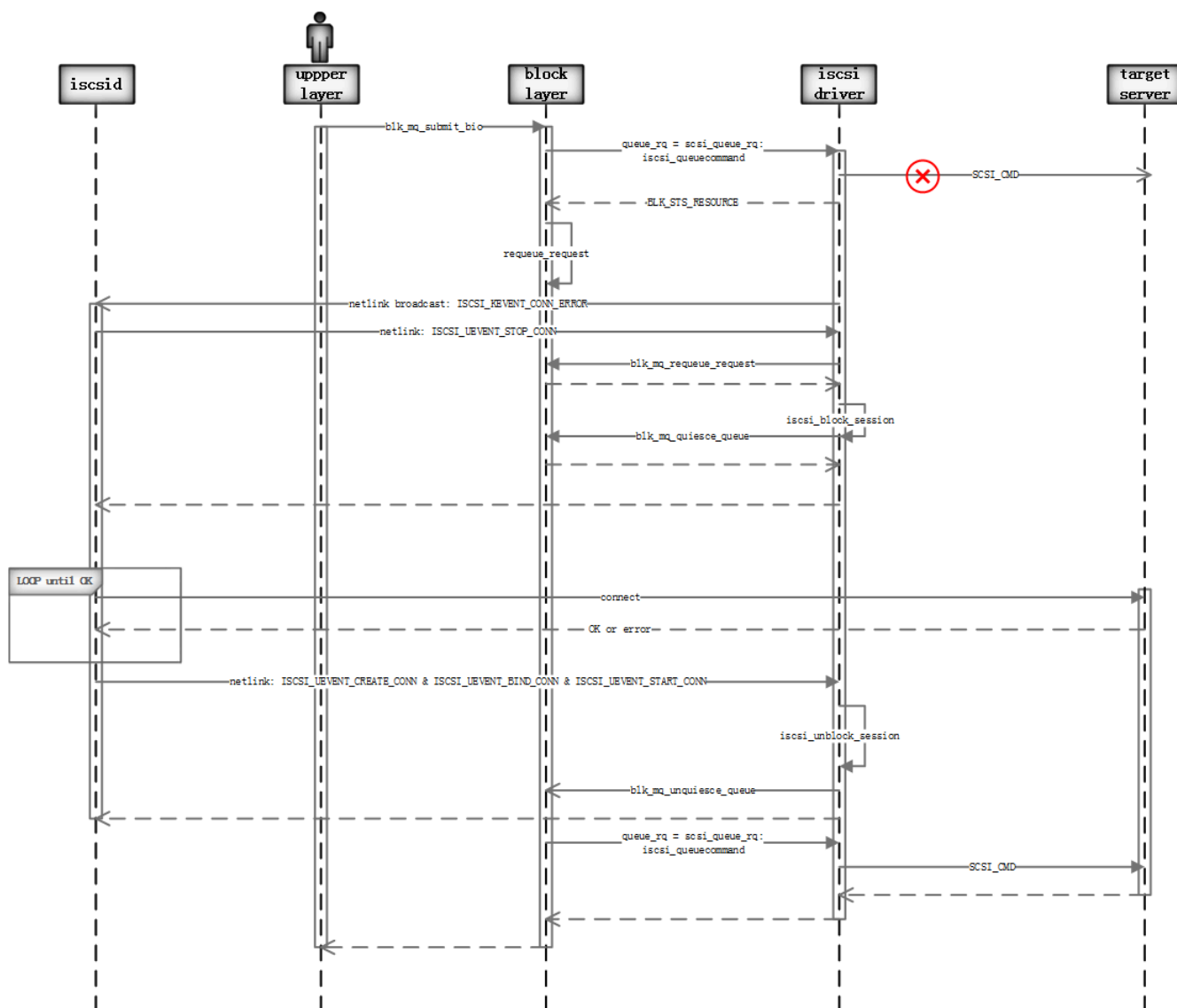
IO:



可以看到，图中大部分消息都是异步消息，和前面的管理平面操作不同的是，数据平面更关注并发要求，所以大部分消息都是异步发送，而不是等待返回之后在发送下一条消息。

(7) client端的NIC socket接收到数据之后，会调用注册的socket iscsi_sw_tcp_data_ready来处理data，不断读取data 到segment，结束之后，调用segment的done函数来处理segment。segment的done函数设置为：iscsi_tcp_hdr_rcv_done，这个函数将解析scsi_cmd的response，设置scsi_cmd的结果，然后put iscsi task。这里如果没有其他人引用task，将进入task的free函数，iscsi_free_task，这个函数将调用scsi_cmd_done，进一步调用块层的 blk_mq_complete_request来完成对应的request。

network Failure:



在了解了iscsi 基本操作和流程之后，重点考察一下网络中断场景iscsi是如何处理的：

(1) IO 流程前半段和普通IO 类似，上层调用`queuecommand`到`iscsi_driver`，然后通过网络发送给target。这个时候如果发送失败，`iscsi_driver` 将会向block 层返回一个`BLK_STS_RESOURCE`，这个时候 block 层会根据这个返回值判断如何处理，这个返回值表示resource busy，那么只需要requeue 就可以了。

补充，这是发现网络故障的一种场景，也就是发送数据的时候失败发现网络故障。但是有可能当前数据已经发送结束了，没有新的数据需要发送，那么这种情况下的网络故障如何探测呢？

实际上在connection setup阶段，启动了一个timer：transport_timer，这个timer会去ping target，如果发现timeout，也会触发`iscsi_conn_failure`。

(2) 返回当前的queuecommand之后，iscsi_driver还会netlink 广播一个connection failure的 kevent，iscsid 会接受处理这个kevent。然后iscsid 向iscsi_driver 发送一个netlink的连接 stop。这个时候iscsi driver 进入到connection stop的流程。流程主要做两件事情，(1) 将当前 driver里面的所有task fail掉，也就是requeue到block 层。(2) 将这个device 的queue 设置成 block状态。防止新的request再加入进来。

(3) iscsid 在stop掉iscsi_driver的连接之后，开始不断的尝试去连接target，一旦成功，会再次通知iscsi_driver去执行start_conn。iscsi_driver就会将queue unblock掉，然后block层就可以继续发送request下来。完成刚才的IO。